**NOTE** Readers who are familiar with the MVC framework may be used to assigning the `Authorize` attribute to action methods within controllers. While `PageModel` handler methods are analogous to controller action methods, Razor Pages does not support assigning the `Authorize` attribute to handler methods. This makes sense if you think about it because you are authorizing the endpoint, regardless of the HTTP method used to access it. A Razor page represents a single endpoint, whereas an MVC controller, more often than not, is responsible for processing multiple endpoints.

At the moment, your Index page returns a `ChallengeResult` to anonymous users if the page is accessed with a `GET` request. You will change that, so anonymous users cannot access the page by protecting it with the `Authorize` attribute. Changes to the code in Pages\Index.cshtml.cs are shown in the next listing.

### Listing 10.2 Applying the `Authorize` attribute to the home `PageModel`

```
using CityBreaks.Models;
using CityBreaks.Services;
using Microsoft.AspNetCore.Authorization;                ❶
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace CityBreaks.Pages
{
    [Authorize]                                          ❷
    public class IndexModel : PageModel
    {
        private readonly ICityService _cityService;

        public IndexModel(ICityService cityService) =>
            _cityService = cityService;

        public List<City> Cities { get; set; }
        public async Task OnGetAsync() =>
            Cities = await _cityService.GetAllAsync();   ❸
    }
}
```

❶ Add a using to bring in the Authorization API.

❷ Add the Authorize attribute to the PageModel class.

❸ Remove the check to see if the user is authenticated.